ANDROID PRACTICE PROJECT REPORT

CryptoApp: Crash Post-Mortem & Architecture Documentation

Complete technical analysis of the Dagger Hilt runtime crash, resolution steps, and architectural verification.

1. Executive Summary & Incident Overview

During the development and testing of **CryptoApp**, navigating to any screen that instantiated a ViewModel triggered an immediate application crash with a fatal runtime exception:

> **FATAL EXCEPTION: main**
> `java.lang.IllegalStateException: Given component holder class com.example.cryptoapp.MainActivity does not implement interface dagger.hilt.internal.GeneratedComponent or interface dagger.hilt.internal.GeneratedComponentManager`
> at dagger.hilt.EntryPoints.get(EntryPoints.java:62)
> at androidx.hilt.lifecycle.viewmodel.compose.HiltViewModelKt.rememberHiltViewModelFactory(HiltViewModel.kt:99)
> at com.example.cryptoapp.AppNavigationKt.AppNavigation(AppNavigation.kt:32)

Primary Impact: The app crashed immediately upon launch as soon as Jetpack Compose attempted to compose `Screen.CoinListScreen` and inject `CoinListViewModel`.

2. Deep-Dive: Root Cause Analysis (What Went Wrong)

To understand why this crash occurred, we must look at how **Dagger Hilt** integrates with **Jetpack Compose** and **Android Architecture Components**:

A. The Dependency Chain

1. **In Composable Screen:** The code calls `val viewModel: CoinListViewModel = hiltViewModel()`.
2. **Context Traversal:** `hiltViewModel()` uses `LocalContext.current` to locate the nearest host `Activity` (in this project, `MainActivity`).
3. **Factory Retrieval:** To instantiate a `@HiltViewModel`, Hilt invokes `HiltViewModelFactory`, which calls `EntryPoints.get(activity, ActivityComponentManager.class)`.
4. **The Missing Interface:** `EntryPoints.get()` asserts that the Activity instance implements `GeneratedComponentManager` or `GeneratedComponent`.

```
[Application] Annotated with @HiltAndroidApp (Initializes SingletonComponent) │ ▼ [MainActivity] Missing @AndroidEntryPoint! ❌ │ ▼ [HiltViewModelFactory] Queries MainActivity for GeneratedComponentManager │ └─→ 💥 CRASH: IllegalStateException (MainActivity does NOT implement GeneratedComponentManager)
```

B. Why was the interface missing?

Hilt relies on bytecode generation via its annotation processor (or KSP). When an Activity is annotated with `@AndroidEntryPoint`, Hilt generates a base class (e.g. `Hilt_MainActivity`) that implements `GeneratedComponentManager` and injects dependencies. Because `MainActivity` lacked `@AndroidEntryPoint`, Hilt treated it as an unmanaged standard Android Activity, causing the type check to fail.

3. The Resolution & Exact Code Modifications

> **The Solution Applied:**
>
> 1. Added `@AndroidEntryPoint` to `MainActivity`.
> 2. Created `CoinApplication` annotated with `@HiltAndroidApp`.
> 3. Registered `CoinApplication` in `AndroidManifest.xml`.
> 4. Added `loadCoin(id)` in `CoinDetailViewModel` and triggered it via `LaunchedEffect(key.id)` in `AppNavigation.kt` to guarantee reliable state loading in Navigation 3.

Modification 1: MainActivity.kt

```
package com.example.cryptoapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import dagger.hilt.android.AndroidEntryPoint

@AndroidEntryPoint // Added: generates ActivityComponent & GeneratedComponentManager
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CryptoAppTheme {
                AppNavigation()
            }
        }
    }
}
```

Modification 2: CoinApplication.kt

```
package com.example.cryptoapp

import android.app.Application
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp // Added: creates the root SingletonComponent for the app lifecycle
class CoinApplication : Application()
```

Modification 3: AndroidManifest.xml

```
<application
    android:name=".CoinApplication"
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.CryptoApp">
    <activity
```

```
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

Modification 4: Navigation 3 & ViewModel Lifecycle Bridge

```
 // In AppNavigation.kt
entry<Screen.CoinDetailScreen> { key ->
    val viewModel: CoinDetailViewModel = hiltViewModel()
    LaunchedEffect(key.id) {
        viewModel.loadCoin(key.id)
    }
    val state = viewModel.state.collectAsState().value
    CoinDetailScreen(uiState = state)
}
```

4. Build & Compilation Verification

The code changes were verified directly using Gradle with the following results:

Verification Step	Command Executed	Outcome
Kotlin & Hilt Compilation	`./gradlew assembleDebug`	**BUILD SUCCESSFUL** (36 tasks executed)
Unit Tests & State Flow	`./gradlew testDebugUnitTest`	**BUILD SUCCESSFUL** (All unit tests passed)
Packaging & APK Generation	`packageDebug`	**Output: app-debug.apk verified**

5. Key Takeaways for Future Android Projects

- **Always Pair Hilt Annotations:** Every app using Hilt must have both an `@HiltAndroidApp` Application class and `@AndroidEntryPoint` on any Activity hosting composables that invoke `hiltViewModel()`.
- **Manifest Registration:** Writing the Application class is not enough; it must always be registered in `AndroidManifest.xml` via `android:name=".CoinApplication"`.
- **Navigation 3 Argument Passing:** In new AndroidX Navigation 3, use `LaunchedEffect(key.property)` to trigger ViewModel queries when passing route parameters from custom `NavKey` objects.

CryptoApp Practice Project • Android Jetpack Compose & Hilt Generated: November 2026